Graph Analysis via Graph Rewriting: Sparse Linear Algebra Based Graph Analysis in Interaction Nets

Danil Zaripov[1,*], Efim Kubishkin[1], Nikolai Ponomarev[1], Georgii Sichkar[1], Vladimir Zaikin[1] and Semyon Grigorev[1]

[1]*Saint Petersburg State University, 7-9 Universitetskaya Embankment, St. Petersburg, 199034, Russia*

Abstract

A clear and well-documented LaTeX document is presented as an article formatted for publication by CEUR-WS in a conference proceedings. Based on the "ceurart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

Keywords

Interaction Nets, graph rewriting, graph analysis algorithms, GraphBLAS, sparse linear algebra, quad-tree

1. Introduction

Graph analysis has become fundamental to modern data science, powering applications ranging from recommendation systems in social networks [?] and fraud detection in financial transactions [?] to protein interaction analysis in bioinformatics [?] and static code analysis [?]. As graph sizes grow to billions of vertices and edges, the need for efficient, scalable implementations becomes critical. However, graph analysis algorithms are difficult to parallelize due to their irregular memory access patterns, poor data locality, and load imbalance across computational units [?].

The GraphBLAS standard [?] addresses these challenges by providing a foundation for constructing high-performance parallel graph analysis algorithms. By expressing graph algorithms in the language of linear algebra—using sparse vectors and matrices as core data structures, together with a rich set of highly parameterizable operations (matrix multiplication, element-wise operations, etc.)—GraphBLAS offers a high-level, mathematically rigorous framework for algorithm construction. Moreover, these linear algebraic primitives can be optimized for diverse hardware architectures, providing a path to high-performance graph analysis across platforms. This approach has proven successful, with implementations like SuiteSparse:GraphBLAS [?] delivering high performance on shared-memory systems, and LAGraph [?] providing a collection of graph analysis algorithms built on these primitives. However, designing parallel functions over sparse data structures remains a nontrivial engineering challenge, particularly for accelerators. Beyond CPU implementations, GraphBLAS has been partially ported to such devices: GraphBLAST [?] and Spla [?] target GPGPUs, while GraphLily [?] targets FPGAs.

Despite its successes, GraphBLAS and its implementations face certain challenges. The algebraic structures central to GraphBLAS—such as semirings and monoids—align more naturally with functional programming paradigms than with imperative ones. Generic data structures, such as matrices and vectors, and functions that required to provide a flexible core for matrix processing, together with user-defined types and operations (which are themselves functions), complicate API design and usage, particularly with respect to type safety. This insight suggests that functional programming languages could simplify the design of GraphBLAS-like APIs, offering cleaner abstractions and more

Woodstock'22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY

*Corresponding author.

kulyabov-ds@rudn.ru (D. Zaripov); st098235@student.spbu.ru (E. Kubishkin); n.ponomarev@spbu.ru (N. Ponomarev); Manfred.Jeusfeld@acm.org (G. Sichkar); Manfred.Jeusfeld@acm.org (V. Zaikin); s.v.grigoriev@mail.spbu.ru (S. Grigorev)

0000-0002-0877-7063 (D. Zaripov); 0009-0009-5531-1758 (E. Kubishkin); 0009-0000-2382-5687 (N. Ponomarev); 0000-0002-9421-8566 (G. Sichkar); 0000-0002-9421-8566 (V. Zaikin); 0000-0002-7966-0698 (S. Grigorev)

© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

composable primitives. However, general-purpose functional languages are often not fast enough for high-performance numerical computing. Research into functional languages and frameworks for parallel computations (including GPUs)—such as Futhark [?], AnyDSL [?], LIFT [?], and Accelerate [?]—while demonstrating promising results, has largely focused on dense computations, leaving sparse irregular workloads less explored.

At the same time, interaction nets [1]—a graph-rewriting model introduced by Yves Lafont—offer a computational model that is inherently well-suited for irregular parallelism. In interaction nets, computation proceeds through local rewriting rules applied to pairs of connected nodes (so-called active pairs). Crucially, multiple non-interfering active pairs can be reduced simultaneously, making interaction nets a promising foundation for parallel irregular computations, including graph analysis. Furthermore, interaction nets are relatively close to functional programming languages, making it easy to transfer design approaches and ideas from the functional world.

In this paper, we present the design and implementation of a GraphBLAS-like linear algebra foundation in two different settings: F# (a mature functional-first language, chosen for ease of prototyping and testing) and Inpla (a language and runtime for interaction nets). We employ quadtrees to represent sparse matrices and vectors, enabling natural divide-and-conquer parallelism through recursive subdivision. On this foundation, we build three canonical graph algorithms—breadth-first search (BFS), single-source shortest paths (SSSP), and triangle counting—using the algebraic primitives provided by our API.

Our contributions are as follows.

1. !!! A design for a GraphBLAS-like API in functional settings, demonstrating how to utilize types for natural graph encoding, together with a minimal set of operations (e.g., `map`, `map2`, `spmv`, `spgemm`).
2. A minimal subset of this API implemented in both F# and Inpla, enabling a direct comparison between a conventional functional language and an interaction net-based approach.
3. An empirical evaluation of BFS, SSSP, and triangle counting using our implementations, analyzing correctness, performance, scalability, and comparison with LAGraph.

The paper is structured as follows. Section 2 provides background on GraphBLAS, quadtree matrix representations, interaction nets, and the Inpla language. Section 3 details our API design and its implementation in both F# and Inpla. Section 4 describes the graph algorithms built on this API. Section 5 presents our experimental setup and evaluation results. Section 6 discusses related work, and Section 7 concludes with directions for future research.

2. Background

2.1. GraphBLAS

Basic ideas.

Problems (implicit zeroes, etc).

SuiteSparse:GraphBLAS as a reference implementation

LAGraph as a collection of graph analysis algorithms (based on SuiteSparse:GraphBLAS).

2.2. Tree-based Representation for Sparse Matrices and Vectors

Explain the quadtree structure. Show how a matrix is recursively subdivided into quadrants (NW, NE, SW, SE) until a uniform or empty quadrant is reached. Contrast this with CSR/CSC formats used in libraries like SuiteSparse. Discuss its advantages (natural divide-and-conquer, adaptive to sparsity patterns) and potential disadvantages (overhead for very regular matrices).

2.3. Interaction Nets

Introduce Interaction Nets as a graph-rewriting model. Explain the core concepts: agents (nodes with a principal port and auxiliary ports), active pairs (two agents connected by their principal ports), and

interaction rules (rewriting rules that define how active pairs are reduced).

Emphasize the key properties: one-step confluence (the order of reduction doesn't matter) and inherent parallelism (active pairs can be reduced simultaneously).

2.4. Inpla

Introduce Inpla: runtime (VM) and language. Briefly describe its syntax for defining agents and rules, and how it allows to program these graph-rewriting systems. This section sets the stage for understanding our Inpla-based implementation.

3. GraphBLAS-like API in Functional style

3.1. Design and Architecture

Architecture, Structure of modules

API: matrices, vectors, functions (map, map2, mxm, mxv, !!!)

3.2. Quad-trees

Quad-trees.

3.3. Interaction Nets

Agents

4. Implemented Graph Analysis Algorithms

implemented algorithms: BFS, SSSP, Triangles counting (name?).

5. Evaluation

5.1. Experimental Setup

Hardware. All experiments were conducted on a system with an AMD Ryzen 9 7900X 12-core processor (24 threads with hyper-threading enabled, base clock 4.7 GHz) and 128 GiB of DDR5 RAM.

Software. We compare four implementations:

- **F#**: Our quadtree-based implementation, compiled with F# !!! and the .NET !!! runtime. This serves as a baseline representing a conventional functional language approach.
- **Inpla**: Our interaction-net-based implementation, executed on the Inpla runtime system (version !!!).
- **LAGraph**: A collection of graph algorithms built on SuiteSparse:GraphBLAS (version !!!), providing state-of-the-art, linear-algebra-based reference implementations of BFS, SSSP, and triangle counting.
- **NetworkX**: A popular Python library for graph analysis (version !!!), included as a baseline representing ease of use over optimization.

Datasets. We evaluate our implementations on several real-world and synthetic graphs from the SuiteSparse Matrix Collection [**?**]. The graphs are chosen to represent a variety of sizes, sparsity patterns, and application domains. Table 1 summarizes their key properties: number of vertices ($|V|$), number of edges ($|E|$), and degree statistics (average, maximum, and median).

Table 1
Dataset properties

Graph	$\|V\|$	$\|E\|$	Avg. degree	Max degree	Median degree
roadNet-CA	!!!	!!!	!!!	!!!	!!!
cit-Patents	!!!	!!!	!!!	!!!	!!!
wikipedia-2007	!!!	!!!	!!!	!!!	!!!
…	…	…	…	…	…

Research Questions. Our evaluation addresses the following research questions:

RQ1 (Correctness): Do the F# and Inpla implementations produce correct results for BFS and triangle counting on all datasets, when compared against reference outputs from NetworkX and LAGraph?

RQ2 (Performance – Inpla vs. State-of-the-Art): How does the execution time of Inpla compare to LAGraph on BFS, SSSP, and triangle counting? We also include F# and NetworkX as additional baselines representing different points in the performance–productivity trade-off.

RQ3 (Parallel Scaling): How well do the Inpla and LAGraph implementations scale with the number of available CPU cores? We measure speedup from 1 to all available cores for BFS, SSSP, and triangle counting, with hyper-threading both enabled and disabled.

RQ4 (Impact of Matrix Reordering): Matrix reordering techniques (e.g., reverse Cuthill–McKee) can improve locality and cache performance. How does applying such reordering affect the performance of Inpla, F#, and LAGraph implementations?

5.2. Evaluation Results

Graphics, tables.

Reordering: figures with sparsity patterns

Analysis.

6. Related Work

Proposed by Lafont, interaction nets can serve as a foundation for irregular parallel computations. Notable implementations include Inpla, HVM, and Vine.

Vine — high-level language (Rust-like) + interaction-nets-based intermediate representation + parallel virtual machine.

HVM — high-level language (JS-like) + interaction-nets-based intermediate representation + parallel virtual machine (+GPGPU?).

Inpla — mature enough, demonstrates good performance and scalability. Interaction-nets-based language + parallel VM. Equipped with a high-level language.

No information is available on the evaluation or comparison of Vine and HVM. Typical benchmarks for these systems include Fibonacci, Ackermann, sorting, and list traversal. No graph analysis algorithms while such algorithms looks grate candidates for interaction nets. Other projects implement interaction nets in languages such as OCaml and Haskell.

Sparse linear algebra and graphs. GraphBLAS API: C API, C++ API, implementations in Java, Julia, etc. Discussion of GraphBLAS problems ("What did GraphBLAS Get Wrong?", John Gilbert, UC Santa Barbara, GraphBLAS BoF at HPEC, September 2022). SuiteSparse:GraphBLAS, LAGraph, GraphBLAS on GPUs. GraphBLAST, GraphLily (FPGA). Manual kernel fusion. Attempts to use MLIR (GraphBLAS dialect). Poor utilization on GPUs. Nontrivial challenge to create generic kernels for GPU and FPGA.

Special hardware for graph analysis. "A Survey on Graph Processing Accelerators: Challenges and Opportunities", "Graph Processing on FPGAs: Taxonomy, Survey, Challenges"

Functional languages for parallel computing. LIFT, AnyDSL, Futhark.

Parallel processors for functional programming languages. Haflang/Heron, Reduceron.

7. Conclusion and Future Work

Summary. Demonstrated a feasible mapping of GraphBLAS concepts to interaction nets using quadtrees. Provided a concrete Inpla implementation and evaluated its scalability.

Bottlenecks and Limitations ??? Duplicator problem: current aggressive strategy limits performance and memory efficiency. ??? Reordering (RCM) is crucial but not automatically handled. Limited set of operations and algorithms evaluated.

Future work.

!!!Improved duplication: Investigate incremental duplication or a custom VM with copy-on-write semantics.!!!

Future work includes extending the set of implemented operations and algorithms. Operations: indexed operations, slices, Algorithms: PageRank, minimum spanning forest (MSF), regular path queries, context-free path queries (CFPQ), and maximal independent set (MIS).

A systematic evaluation of other interaction-net-based implementations, such as HVM and Vine, is also required.

Alternative sparse matrix representations, such as the coordinate list (COO) format, rely on sorting—an operation that parallelizes well in interaction nets and therefore warrants further investigation.

Kernel fusion presents another direction. Techniques such natural for functional programming advanced optimization techiques as supercompilation [?] or distillation [?] could be applied to automatically fuse sequences of operations, analogous to manual fusion for masks and other constructs in classic GraphBLAS. Can they be used for interaction nets?

Finally, hardware acceleration remains an open opportunity. Designing an FPGA specifically optimized for interaction net reduction could exploit the fine-grained parallelism inherent to the model and yield substantial performance gains.

8. Code Availability Statement

F# prototyping code.

Inpla implementation.

Benchmarking scripts and dataset configurations.

Acknowledgments

Thanks to the developers of ACM consolidated LaTeX styles https://github.com/borisveytsman/acmart and to the developers of Elsevier updated LaTeX templates https://www.ctan.org/tex-archive/macros/latex/contrib/els-cas-templates.

Declaration on Generative AI

During the preparation of this work, the author(s) used X-GPT-4 and Gramby in order to: Grammar and spelling check. Further, the author(s) used X-AI-IMG for figures 3 and 4 in order to: Generate images. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

References

[1] Y. Lafont, Interaction nets, in: Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '90, Association for Computing Machinery, New York, NY, USA, 1989, p. 95–108. URL: https://doi.org/10.1145/96709.96718. doi:10.1145/96709.96718.